MINI EMAIL SYSTEM

A PROJECT REPORT

Submitted by

JAYASAKTHI R (8115U24AM027)

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**)**

K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
SAMAYAPURAM, TRICHY

ANNA UNIVERSITY
CHENNAI 600 025

MAY 2026

MINI EMAIL SYSTEM

PROJECT FINAL DOCUMENT

Submitted by

JAYASAKTHI R (8115U24AM027)

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**)**

Under the Guidance of

Mr. P. NAGARAJ M.E.,

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled **"MINI EMAIL SYSTEM"** is the bonafide work of **JAYASAKTHI R (8115U24AM027)** who carried out the work under my supervision.

SIGNATURE	**SIGNATURE**
Dr. B. KIRAN BALA M.E.,M.B.A.,Ph.D.,	**Mr. P. NAGARAJ M.E.,**
HEAD OF THE DEPARTMENT ASSOCIATE PROFESSOR,	**SUPERVISOR ASSISTANT PROFESSOR,**
Department of Artificial Intelligence and Machine Learning, K. Ramakrishnan College of Engineering, (Autonomous) Samayapuram, Trichy.	Department of Artificial Intelligence and Machine Learning, K. Ramakrishnan College of Engineering, (Autonomous) Samayapuram, Trichy.

SIGNATURE OF INTERNAL EXAMINER	**SIGNATURE OF EXTERNAL EXAMINER**
NAME:	**NAME:**
DATE:	**DATE:**

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

DECLARATION BY THE CANDIDATE

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the project Viva-Voice held at K. Ramakrishnan College of Engineering on _____

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I thank the almighty GOD, without whom it would not have been possible for me to complete my project.

I wish to address my profound gratitude to **Dr.K.RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering(Autonomous), who encouraged and gave me all help throughout the course.

I extend my hearty gratitude and thanks to my honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.,** K. Ramakrishnan College of Engineering(Autonomous).

I am glad to thank my Principal **Dr.D.SRINIVASAN, M.E., Ph.D.,FIE., MIIW., MISTE., MISAE., C.Engg,** for giving me permission to carry out this project.

I wish to convey my sincere thanks to **Dr.B.KIRAN BALA, M.E., M.B.A., Ph.D.,** Head of the Department, Artificial Intelligence and Machine Learning for giving me constant encouragement and advice throughout the course.

I am grateful to **Mr.P.NAGARAJ, M.E., Assistant Professor**, Artificial Intelligence and Machine Learning, K. Ramakrishnan College of Engineering (Autonomous), for his guidance and valuable suggestions during the course of study.

Finally, I sincerely acknowledged in no less terms all my staff members, my parents and, friends for their co-operation and help at various stages of this project work.

JAYASAKTHI R (8115U24AM027)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

VISION

To become a renowned hub for AIML technologies to producing highly talented globally recognizable technocrats to meet industrial needs and societal expectation.

MISSION

Mission of the Department

M1 To impart advanced education in AI and Machine Learning, built upon a foundation in Computer Science and Engineering.

M2 To foster Experiential learning equips students with engineering skills to tackle real-world problems.

M3 To promote collaborative innovation in AI, machine learning, and related research and development with industries.

M4 To provide an enjoyable environment for pursuing excellence while upholding strong personal and professional values and ethics.

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

PEO1 Excel in technical abilities to build intelligent systems in the fields of AI & ML in order to find new opportunities.

PEO2 Embrace new technology to solve real-world problems, whether alone or as a team, while prioritizing ethics and societal benefits.

PEO3 Accept lifelong learning to expand future opportunities in research and product development.

PROGRAM SPECIFIC OUTCOMES (PSO's)

PSO1 Expertise in tailoring ML algorithms and models to excel in designated applications and fields.

PSO2 Ability to conduct research, contributing to machine learning advancements and innovations that tackle emerging societal challenges.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO8 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO9 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO10 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO11 Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

The **Mini Email System** is a Python-based application developed using Streamlit and MySQL to provide a simple, secure, and efficient platform for managing email communication within a system. The application allows users to register, log in, send emails, receive messages, and manage their inbox in a structured and organized manner. It ensures proper data management by storing user details and email records in relational database tables, reducing redundancy and maintaining data consistency. The system supports essential features such as user authentication, email composition, inbox viewing, and soft deletion of emails using database operations like INSERT, SELECT, and UPDATE. The application is designed with an interactive and user-friendly interface, enabling users to easily navigate between modules such as login, sending emails, and viewing received messages. SQL queries are used for efficient data storage and retrieval, ensuring fast response time and reliable system performance. The database design follows normalization principles to improve efficiency and avoid data anomalies. Overall, the system demonstrates the practical implementation of database management system concepts in a real-time communication environment, providing a strong foundation for developing advanced email systems with additional features such as attachments, notifications, and cloud integration.

ABSTRACT WITH POs AND PSOs MAPPING

ABSTRACT	POs MAPPED	PSOs MAPPED
The The Mini Email System is a Python-based application developed using Streamlit and MySQL to provide a simple, secure, and efficient platform for managing email communication within a system. The application allows users to register, log in, send emails, receive messages, and manage their inbox in a structured and organized manner. It ensures proper data management by storing user details and email records in relational database tables, reducing redundancy and maintaining data consistency. The system supports essential features such as user authentication, email composition, inbox viewing, and soft deletion of emails using database operations like INSERT, SELECT, and UPDATE. The application is designed with an interactive and user-friendly interface, enabling users to easily navigate between modules such as login, sending emails, and viewing received messages. SQL queries are used for efficient data storage and retrieval, ensuring fast response time and reliable system performance.	**PO1-3** **PO2-3** **PO3-3** **PO4-3** **PO5-3** **PO6-3** **PO7-3** **PO8-3** **PO9-3** **PO10-3** **PO11-3**	**PSO1 – 3** **PSO2 – 3**

Note: 1- Low, 2-Medium, 3- High

TABLE OF CONTENTS

CHAPTER No.		TITLE	PAGE No.
		ABSTRACT	**vi**
		LIST OF ABBREVIATIONS	**xi**
		LIST OF FIGURES	**xii**
1		**INTRODUCTION**	**1**
	1.1	Objective	1
	1.2	Scope of the Project	2
	1.3	Overview of the System	3
	1.4	Importance of DBMS in Email system	3
	1.5	Technologies Used	5
2		**PROJECT METHODOLOGY**	**7**
	2.1	Requirement Analysis	7
	2.2	Entity-Relationship (ER) Diagram	9
	2.3	Relational Schema	10
	2.4	Normalization Process	11
	2.5	System Architecture Overview	12
3		**MODULE DESCRIPTION**	**14**
	3.1	User Registration Module	14
	3.2	Existing User Validation Module	15
	3.2	User Login Module	15
	3.3	Send Email Module	16
	3.4	Inbox Email Module	16
	3.5	Delete Email Module	17
	3.6	Logout Module	18
4		**DATABASE IMPLEMENTATION**	**19**
	4.1	Table Creation Scripts	19
	4.2	Constraints and Relationships	20

	4.3	Backup & Recovery Considerations	21
5		**RESULTS AND DISCUSSION**	**22**
	5.1	Performance Analysis	22
	5.2	Observations and Interpretations	23
6		**CONCLUSION AND FUTURE WORK**	**25**
	6.1	Summary of Outcomes	25
	6.2	Future Scope and Enhancements	26
7		**APPENDIX**	**27**
		APPENDIX A – Source Code	27
		APPENDIX B - Screenshots	33
		REFERENCES	**36**

LIST OF ABBREVIATIONS

S. NO.	ACRONYM	ABBREVIATION
1	API	Application Programming Interface
2	CRUD	Create, Read, Update, Delete
3	DBMS	Database Management System
4	ER	Entity Relationship
5	IDE	Integrated Development Environment
6	RDBMS	Relational Database Management System
7	SMTP	Simple Mail Transfer Protocol
9	SQL	Structured Query Language
10	UI	User Interface

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
2.2.1	Entity Relationship	9
2.3.1	Relational Schema	10
2.5.1	System Architecture	13
B.1	User Registration Page	34
B.2	User Login Page	34
B.3	Inbox Display	35
B.4	Send Email Interface	35
B.5	Delete Email Module	36
B.6	Duplicate Email Validation during User Registration	36

CHAPTER 1
INTRODUCTION

1.1 OBJECTIVE

The primary objective of the Mini Email System is to design and develop a simple, secure, and efficient communication platform that enables users to send, receive, and manage emails using a structured database system. In the modern digital world, communication plays a vital role in both personal and professional environments, and email systems are one of the most widely used communication tools. Therefore, it is essential to develop a system that can handle user interactions and message storage in an organized and reliable manner.

This project aims to provide a centralized platform where users can register, log in securely, and exchange messages within the system. By implementing a database-driven approach, the system ensures that all user and email data are stored systematically, reducing redundancy and maintaining consistency. The application supports core functionalities such as user authentication, email composition, inbox management, and email deletion, which are fundamental to any email system.

Another important objective of this project is to demonstrate the practical implementation of Database Management System (DBMS) concepts such as relational schema design, normalization, and the use of SQL queries for performing CRUD (Create, Read, Update, Delete) operations. The system also ensures that duplicate user accounts are avoided by validating existing users before registration, thereby maintaining data integrity.

The project is designed to be user-friendly, allowing users to easily navigate through different modules without technical complexity. It also focuses on providing a responsive interface and efficient backend processing to ensure smooth system performance. Overall, the objective of this project is to build a functional mini email system that serves as a foundation for understanding real-world communication systems and database applications.

1.2 SCOPE OF THE PROJECT

The Mini Email System is designed to provide a basic yet efficient platform for managing email communication using database concepts. The scope of this project includes essential functionalities such as user registration, login authentication, sending emails, receiving messages, and managing inbox data. It is primarily intended for academic purposes and serves as a demonstration of how database systems can be used to build real-time communication applications.

The system allows users to create accounts by providing necessary details such as name, email, and password. Once registered, users can log in securely and access the system features. After successful login, users can send emails to other registered users by entering the receiver's email address, subject, and message content. The system stores all emails in a database, enabling users to view their received messages in the inbox module.

The project also includes a delete functionality that allows users to remove emails from their inbox using a soft delete mechanism. Instead of permanently deleting records, the system updates the status of emails, ensuring data safety and recoverability. Additionally, the system prevents duplicate user registrations by checking whether an email already exists in the database before allowing account creation.From a technical perspective, the system is implemented using Python (Streamlit) for the user interface and MySQL as the backend database. The integration of these technologies ensures efficient data handling, fast query execution, and a user-friendly experience.

The scope of the project can be extended in the future by adding advanced features such as email attachments, real-time notifications, spam filtering, encryption for security, and cloud-based storage. It can also be enhanced to support large-scale users and integrated with mobile applications. Overall, the project provides a strong foundation for developing more advanced communication systems.

1.3 OVERVIEW OF THE SYSTEM

The Mini Email System is a database-driven application designed to facilitate communication between users in a simple and efficient manner. The system provides an interactive interface where users can perform operations such as account registration, login, sending emails, viewing inbox messages, and deleting emails.

The system follows a structured architecture consisting of three main components: the front-end interface, the application logic, and the database. The front-end is developed using Streamlit, which provides an easy-to-use and interactive interface for users. The application logic is implemented using Python, which handles user inputs, processes requests, and communicates with the database. The backend database is managed using MySQL, where all user and email data are stored in a structured format.

The system operates in a workflow where users first register and log in to access the application. Once logged in, users can send emails by entering the recipient's email address, subject, and message. These emails are stored in the database and can be retrieved by the receiver in their inbox. The inbox module displays all received emails, allowing users to read and manage their messages effectively.

The system also includes a delete functionality, where emails are not permanently removed but are marked as deleted using a boolean field. This approach ensures that data integrity is maintained and allows for possible recovery if needed. The system ensures that all interactions between users and the database are handled efficiently using SQL queries.

Overall, the Mini Email System provides a simple, reliable, and scalable solution for managing email communication. It demonstrates how database systems can be integrated with application logic to create real-world applications. The modular design of the system allows for easy maintenance and future enhancements.

1.4 IMPORTANCE OF DBMS IN LIBRARY SYSTEMS

A Database Management System (DBMS) plays a crucial role in the development and functioning of the Mini Email System. Since the application involves handling user data and email messages, it is essential to store and manage this information in an organized and efficient manner. A DBMS provides the necessary tools and functionalities to handle large amounts of data while ensuring accuracy, consistency, and security.

One of the key roles of DBMS in this system is data organization. User details and email messages are stored in separate tables with proper relationships, making it easy to manage and retrieve data. This structured approach helps in avoiding confusion and improves system efficiency. For example, user information is stored in the Users table, while email data is stored in the Emails table, ensuring clear separation of data.

Another important aspect is data integrity. The DBMS enforces constraints such as primary keys, foreign keys, and unique constraints to ensure that the data stored in the database is accurate and consistent. It prevents duplicate entries, such as multiple accounts with the same email address, and ensures that all email records are properly linked to valid users.

DBMS also enables efficient data retrieval using SQL queries. Users can quickly access their inbox messages, and the system can perform operations such as inserting new emails, updating email status, and retrieving user information with minimal delay. This improves overall system performance and user experience.

Security is another critical factor where DBMS plays an important role. Sensitive information such as user credentials is stored securely, and access is restricted to authorized users only. The system ensures that only authenticated users can access their respective data, thereby protecting user privacy.

Additionally, DBMS supports backup and recovery mechanisms, which help in preventing data loss due to system failures or errors. The use of normalization techniques reduces data redundancy and improves storage efficiency.Overall, the DBMS serves as the backbone of the Mini Email System, ensuring efficient data management, security, and reliability.

1.5 TECHNOLOGIES USED

The Mini Email System is developed using a combination of modern technologies that work together to provide a reliable, efficient, and user-friendly application. These technologies are categorized into front-end, back-end, database, and development tools.The front-end of the application is developed using Python Streamlit, which provides an interactive and easy-to-use interface for users. Streamlit allows the creation of web-based applications with minimal code, making it suitable for developing simple and efficient user interfaces. It enables the use of components such as text inputs, buttons, and menus, which help users interact with the system effectively.

The back-end of the system is implemented using Python, which handles all the application logic and processing. Python is responsible for managing user authentication, sending emails, retrieving inbox messages, and interacting with the database. It ensures that all operations are performed correctly and efficiently.

The database used in this project is MySQL, which is a relational database management system. It stores all user and email data in structured tables, ensuring efficient data storage and retrieval. SQL is used to perform various database operations such as inserting new records, retrieving data, updating information, and deleting records.

In addition to these core technologies, development tools such as Visual Studio Code and MySQL Workbench are used for coding and database management. These tools provide a convenient environment for writing code, testing the application, and managing the database. The integration of these technologies ensures smooth

communication between the user interface, application logic, and database. This results in a system that is efficient,Sensitive data and system access are controlled using authentication mechanisms, ensuring that only authorized users can perform operations like updating or deleting records.In case of unexpected failures, the DBMS ensures reliable data storage and supports recovery mechanisms to prevent data loss.

As the number of books and users increases, the DBMS can handle large volumes of data efficiently. Indexing and optimized queries help maintain system performance.The system can support advanced features such as search functionality, filtering, and reporting by using SQL queries and database operations.The database is normalized to reduce redundancy and improve performance. Proper schema design ensures faster query execution and efficient resource utilization.

In summary, the DBMS serves as the backbone of the Library Management System, enabling it to function smoothly and efficiently. It ensures that all data-related operations such as book management, member tracking, and transaction handling are performed reliably and securely. Without a robust DBMS, the system would face challenges in maintaining data consistency, security, scalability, and performance.

CHAPTER 2

LITERATURE SURVEY

This chapter describes the overall development approach of the Mini Email System, including requirement analysis, database design, normalization, and system architecture. It provides a strong foundation for building an efficient and reliable application by applying core Database Management System (DBMS) concepts and design methodologies. The chapter focuses on identifying system requirements, defining entities and relationships, and organizing data in a structured manner to ensure consistency and integrity.

2.1 REQUIREMENT ANALYSIS

Requirement analysis is one of the most important phases in the development of any system. It involves identifying the needs of the users and defining the functionalities required for the system to operate effectively. For the Mini Email System, the requirements are categorized into functional and non-functional requirements to ensure clarity and completeness in system design.

Functional Requirements:

The Mini Email System includes several core functionalities that enable users to interact with the system efficiently. These include:

- **User Registration:** The system must allow new users to create an account by providing details such as name, email, and password. It should also validate whether the email already exists to prevent duplicate registrations.

- **User Login:** Registered users must be able to log in securely using their email and password. The system verifies credentials from the database before granting access.

- **Send Email:** Users should be able to compose and send emails by entering the

receiver's email address, subject, and message.

- **Inbox Management:** The system must display all received emails in an organized format, allowing users to read messages easily.

- **Delete Email:** Users should be able to delete emails from their inbox using a soft delete mechanism, where the email is marked as deleted instead of being permanently removed.

- **Session Management:** The system should maintain user session data to ensure continuous access until logout.

Non-Functional Requirements:

- **Usability:** The system should provide a simple and user-friendly interface for easy navigation.

- **Performance:** The application should respond quickly to user actions such as login, sending emails, and viewing inbox.

- **Security:** User data, especially login credentials, must be protected and accessible only to authorized users.

- **Reliability:** The system should function without crashes and handle errors effectively.

- **Scalability:** The system should be capable of handling an increasing number of users and emails.

- **Maintainability:** The system should be easy to update and modify for future enhancements.

2.2 ENTITY-RELATIONSHIP (ER) DIAGRAM

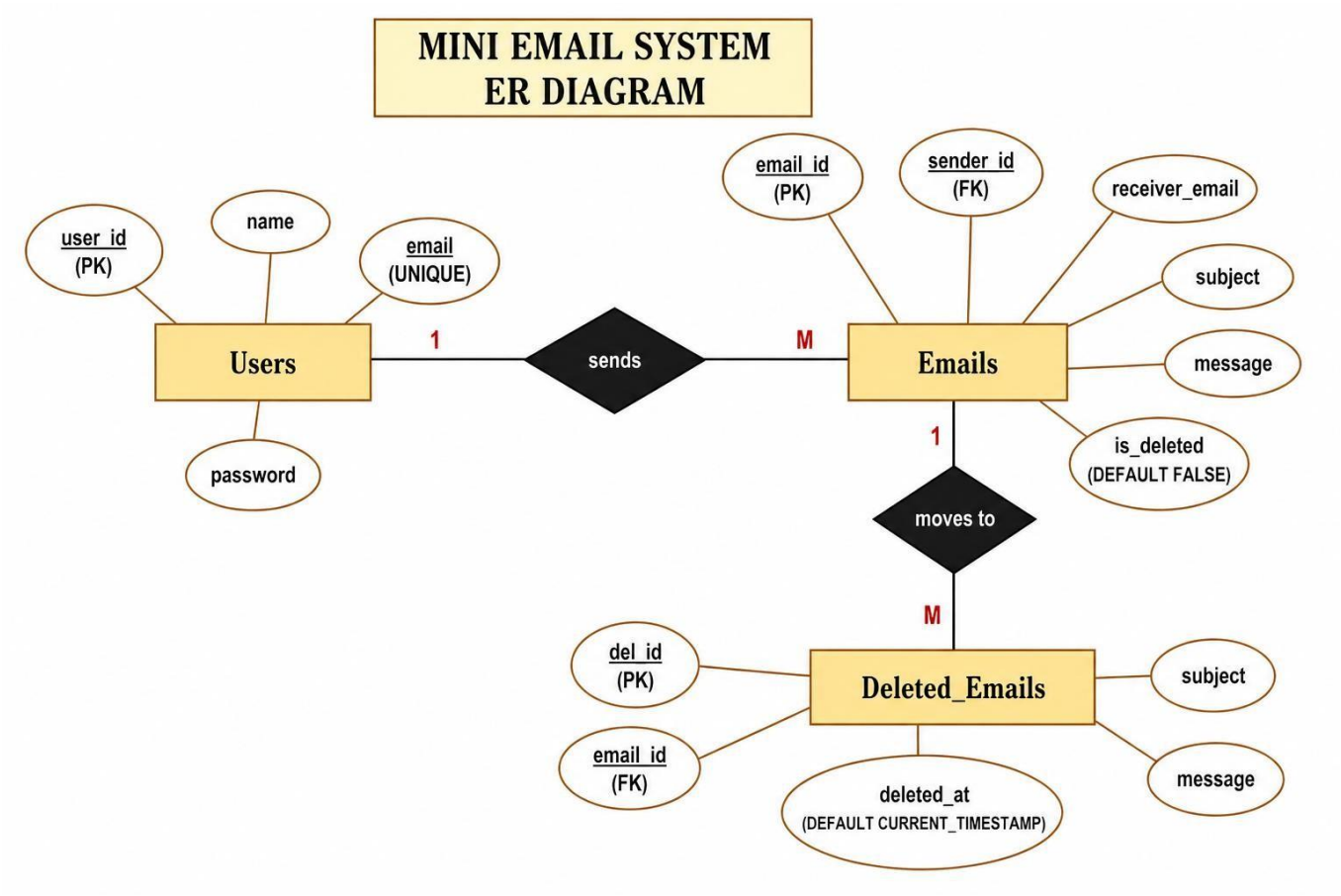

Figure 2.2.1 Entity Relationship

The ER Diagram illustrates how different entities in the Mini Email System are connected. It provides a blueprint for structuring the database.

Main Entities:

Users:

Stores user details such as user ID, name, email, and password, which are used for authentication and identification

Emails:

Contains email information such as email ID, sender ID, receiver ID, subject, message, and deletion status, which are used for managing communication between users

Relationships:

Sends:

Represents the relationship between Users and Emails, where one user can send multiple emails (one-to-many relationship)

Receives:

Represents the relationship between Users and Emails, where one user can receive multiple emails (one-to-many relationship)

Each email is associated with one sender and one receiver, ensuring proper linkage between users and emails

2.3 RELATIONAL SCHEMA

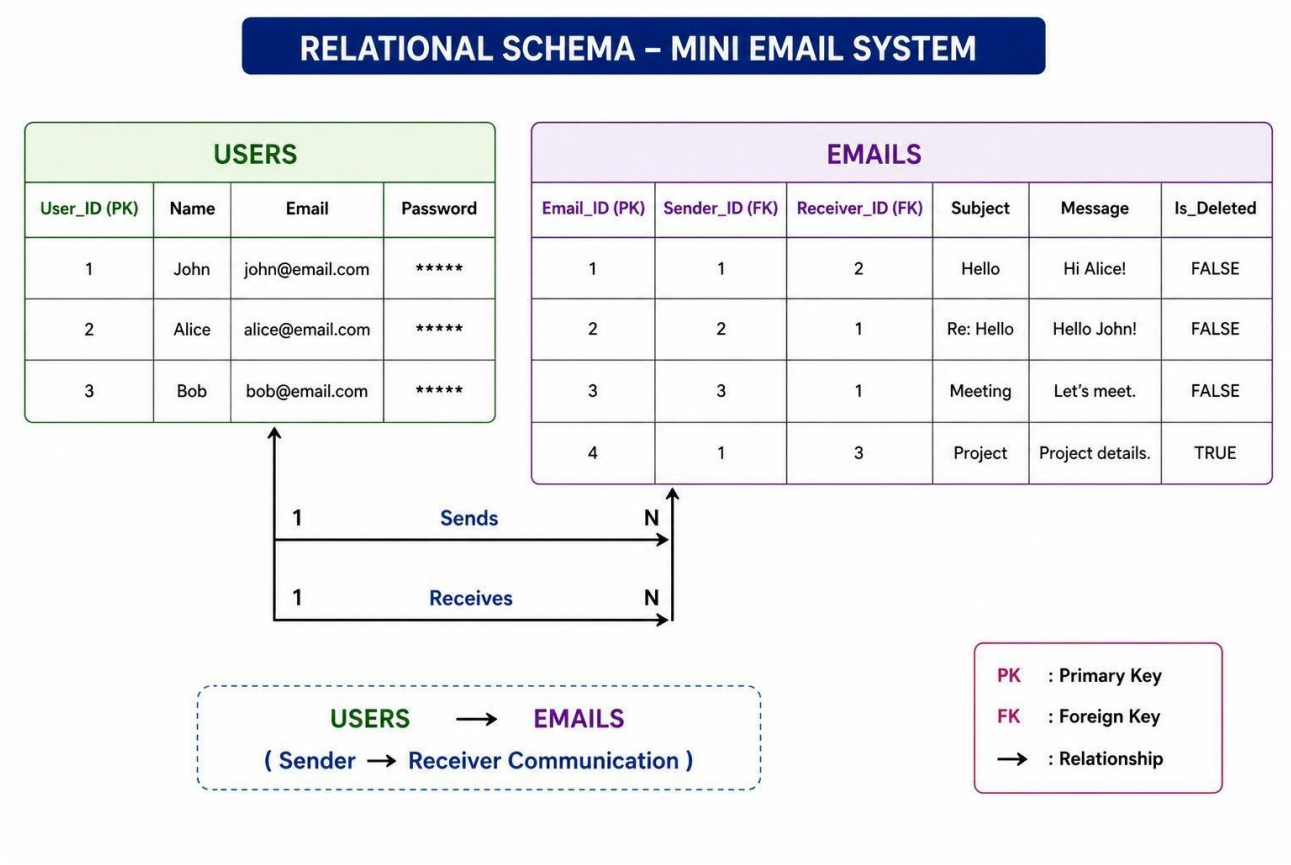

Figure 2.3.1 Relational Schema

The relationship between the entities in the Mini Email System is defined by the "Sends" and "Receives" relationships between Users and Emails. A user can send multiple emails, establishing a one-to-many relationship between Users and Emails through the sender_id attribute. Similarly, a user can receive multiple emails, forming another one-to-many relationship through the receiver_id attribute. Each email is associated with exactly one sender and one receiver, ensuring proper linkage and data consistency. These relationships help in maintaining structured communication flow within the system and ensure that every email is correctly mapped to its respective users.

2.4 NORMALIZATION PROCESS

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It ensures that data is stored efficiently and minimizes duplication.

The Mini Email System follows the first three normal forms:

First Normal Form (1NF):

Ensures that all attributes contain atomic values and there are no repeating groups. For example, each email record contains a single subject and message.

Second Normal Form (2NF):

Ensures that all non-key attributes are fully dependent on the primary key. In this system, all attributes in the Users and Emails tables depend on their respective primary keys.

Third Normal Form (3NF):

Ensures that there are no transitive dependencies. This means non-key attributes depend only on the primary key and not on other non-key attributes.

By applying normalization, the system avoids data duplication and ensures consistency across the database. For example, user information is stored only in the Users table and not repeated in the Emails table.

2.5 SYSTEM ARCHITECTURE OVERVIEW

The Mini Email System follows a three-tier architecture, which consists of the Presentation Layer, Application Layer, and Data Layer. This layered approach improves system organization, scalability, and maintainability.

Presentation Layer (Frontend):

- This layer is responsible for user interaction and is developed using Streamlit.
- It provides an interface where users can perform actions such as registration, login, sending emails, and viewing inbox messages.
- The interface is designed to be simple and user-friendly, ensuring smooth navigation.

Application Layer (Backend):

- The application layer is implemented using Python and handles all business logic.
- It processes user inputs, performs validations, executes database queries, and manages system operations.
- It acts as a bridge between the frontend and the database.

Data Layer (Database):

- The data layer consists of the MySQL database, where all data is stored in structured tables.
- It manages user information and email records using SQL queries.

- The database ensures data integrity, consistency, and efficient retrieval.

Data Flow:

1. User interacts with the frontend
2. Request is sent to the backend
3. Backend processes the request
4. Data is stored/retrieved from the database
5. Result is displayed to the user

The architecture ensures efficient communication between components and provides a strong foundation for system scalability and future enhancements.

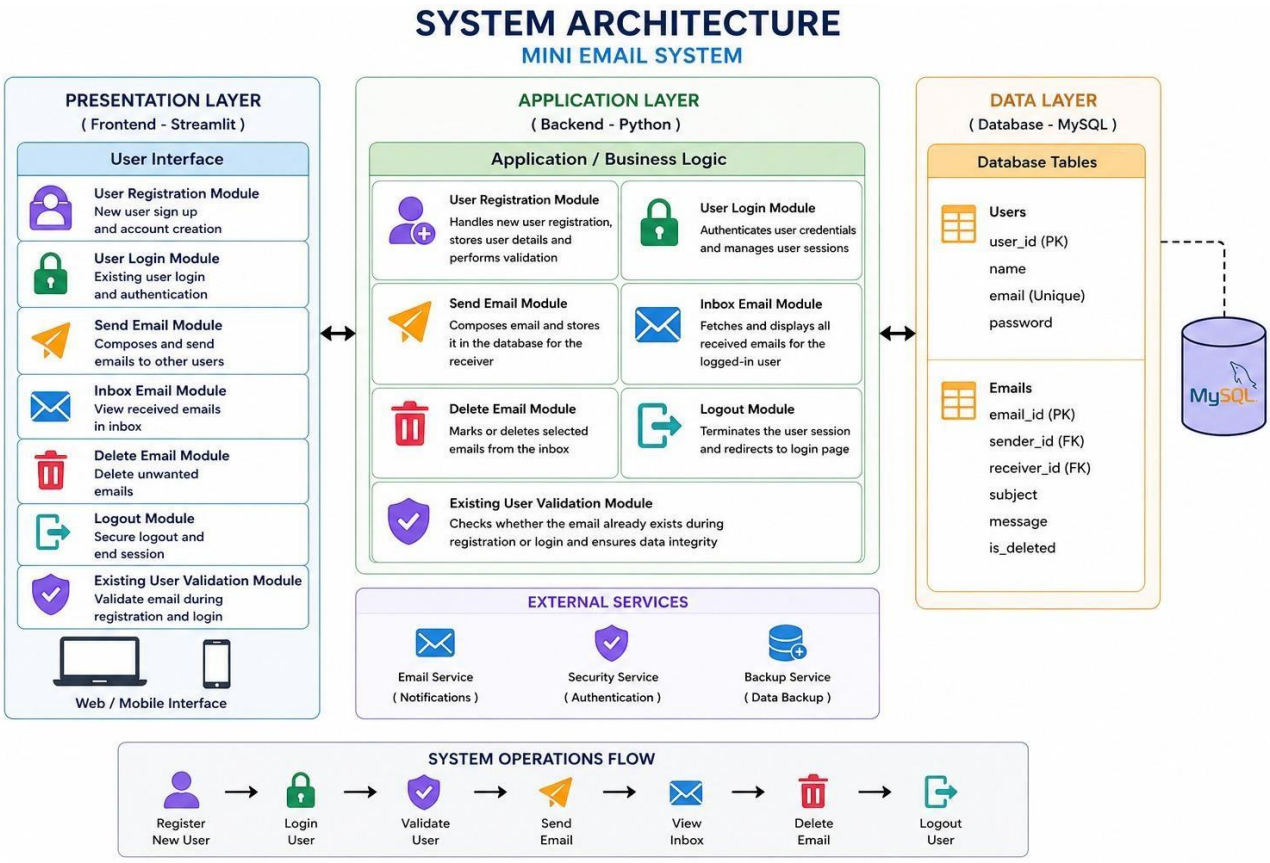

Figure 2.5.1 System Architecture

CHAPTER 3

PROJECT METHODOLOGY

This chapter describes the different modules of the Mini Email System, which are designed to perform specific tasks and ensure smooth functioning of the application. The modular approach helps in dividing the system into smaller components, making it easier to develop, maintain, and enhance. Each module is responsible for a particular functionality such as user authentication, email handling, and data management. These modules work together to provide a complete and efficient email communication system.

3.1 USER REGISTRATION MODULE

The User Registration Module allows new users to create an account in the system. It ensures that user details such as name, email, and password are collected and stored securely in the database. This module plays a crucial role in maintaining user records and enabling access to the system.

Key Features:

- Account creation using name, email, and password
- Stores user data in database
- Ensures secure user registration

Functionality Flow:

1. User enters name, email, and password
2. System validates input fields
3. Data is sent to database
4. User account is created successfully

3.2 EXISTING USER VALIDATION MODULE

The Existing User Validation Module checks whether a user email already exists in the database before allowing registration. This module prevents duplicate user accounts and maintains data integrity.

Key Features:

- Checks for existing email in database
- Prevents duplicate registration
- Displays error message if email exists

Functionality Flow:

1. User enters email during signup
2. System queries database for existing email
3. If email exists → Error displayed
4. If not → Registration continues

3.3 USER LOGIN MODULE

The User Login Module allows registered users to access the system securely by verifying their credentials. It ensures that only authorized users can log in and use the system features.

Key Features:

- Secure login authentication
- Verifies email and password
- Maintains user session

Functionality Flow:

1. User enters email and password
2. System checks credentials in database
3. If valid → Login successful
4. If invalid → Error message displayed

3.4 SEND EMAIL MODULE

The Send Email Module enables users to compose and send emails to other users within the system. It allows users to enter receiver details and message content, which are stored in the database.

Key Features:

- Compose email with subject and message
- Send email to receiver
- Stores email data in database

Functionality Flow:

1. User enters receiver email, subject, and message
2. System validates input
3. Email details are stored in database
4. Confirmation message is displayed

3.5 INBOX MODULE

The Inbox Module displays all emails received by the logged-in user. It retrieves data from the database and presents it in a readable format.

Key Features:

- Displays received emails

- Fetches data from database
- Shows subject and message

Functionality Flow:

1. User opens inbox
2. System fetches emails from database
3. Emails are filtered based on user
4. Emails are displayed on screen

3.6 DELETE EMAIL MODULE

The Delete Email Module allows users to remove emails from their inbox. Instead of permanently deleting the email, the system uses a soft delete mechanism by updating the email status.

Key Features:

- Soft delete functionality
- Updates email status
- Maintains data integrity

Functionality Flow:

1. User enters email ID
2. System updates is_deleted field
3. Email is hidden from inbox
4. Confirmation message displayed

3.7 LOGOUT MODULE

The Logout Module allows users to securely exit the system. It clears the session data and redirects the user to the login page.

Key Features:

- Ends user session
- Prevents unauthorized access
- Redirects to login page

Functionality Flow:

1. User clicks logout
2. Session data is cleared
3. User is redirected to login page

CHAPTER 4
DATABASE IMPLEMENTATION

This chapter describes the database implementation of the Mini Email System, which plays a crucial role in storing and managing user and email data in a structured format. The database is designed using a relational model to ensure efficient data organization, integrity, and consistency. It includes the creation of tables, defining relationships between them, and applying constraints such as primary keys, foreign keys, and unique attributes. The implementation also focuses on efficient data storage and retrieval using SQL queries. In addition, considerations for backup and recovery are included to ensure data safety and reliability. Overall, this chapter highlights how the database is designed and implemented to support the smooth functioning of the Mini Email System.

4.1 TABLE CREATION SCRIPTS

The Mini Email System uses a relational database management system such as MySQL to store and manage user and email data efficiently. The database is designed using structured tables with proper relationships to ensure data integrity and easy retrieval. The following tables form the core of the system:

1. Users Table

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(50)
);
```

- **Purpose:** The Users table stores all registered user details such as user ID, name, email, and password. The email field is defined as UNIQUE to prevent duplicate user accounts.

2. Emails Table

```
CREATE TABLE Emails (
    email_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_id INT,
    receiver_email VARCHAR(100),
    subject VARCHAR(100),
    message TEXT,
    is_deleted BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (sender_id) REFERENCES Users(user_id)
);
```

- **Purpose:** The Emails table stores all email messages sent by users. It includes sender information, receiver email, subject, message, and deletion status.

4.2 CONSTRAINTS AND RELATIONSHIPS

Implementing primary keys, foreign keys, and constraints ensures that the database maintains data integrity, avoids redundancy, and enforces logical connections between tables.

- **Primary Keys:**

  Primary keys uniquely identify each record in a table (e.g., user_id, email_id). They ensure that no duplicate records exist and each entry can be accessed directly. Primary keys are automatically indexed, which improves data retrieval speed.

- **Foreign Keys:**

  Foreign keys establish relationships between different tables in the database. In this system, sender_id in the Emails table links to user_id in the Users table. They ensure referential integrity, meaning related data must exist in the parent table. Prevents invalid data entry (e.g., an email cannot exist without a valid user).

- **Constraints:**

  NOT NULL: Ensures important fields like email and password are always filled. **UNIQUE:** Prevents duplicate values (e.g., user email must be unique). **DEFAULT:** Assigns default values automatically (e.g., is_deleted = FALSE). **CHECK (optional):** Ensures valid data (e.g., message should not be empty). **ON UPDATE CASCADE (optional):** Updates related records automatically when key values change.

4.3 BACKUP AND RECOVERY CONSIDERATIONS

Backup and recovery are essential components of any database system to ensure data safety and reliability. In the Mini Email System, backup refers to the process of creating a copy of the database that contains all user information and email records. This backup can be used to restore the system in case of data loss, system failure, or unexpected errors. Regular backups help in protecting important data and maintaining system continuity.In this system, backups can be performed using MySQL tools such as exporting the database into SQL dump files. Backup operations can be scheduled periodically to ensure that the most recent data is always available.

Recovery is the process of restoring the database from a previously created backup. In case of system crashes, accidental deletion of records, or data corruption, the backup file can be used to recreate the database and restore it to its previous state. This ensures that user accounts and email data are not permanently lost.The Mini Email System also supports data safety through the use of a soft delete mechanism, where emails are not permanently removed but are marked as deleted using the is_deleted attribute.

Overall, the implementation of backup and recovery mechanisms improves the reliability, availability, and fault tolerance of the system, ensuring that data is protected and can be restored whenever necessary.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the results obtained from the implementation of the Mini Email System and discusses the performance and behavior of the system. The application was developed successfully using Python (Streamlit) and MySQL, and all the core functionalities were tested to ensure proper working of the system. The results show that the system is able to handle user registration, login, sending emails, receiving messages, and deleting emails efficiently.

5.1 PERFORMANCE ANALYSIS

The performance of the Mini Email System was evaluated based on key aspects such as response time, database operations, user interface responsiveness, and overall system reliability.

Frontend Performance:

- The Streamlit-based interface is responsive and easy to use.
- Navigation between login page, inbox, and email modules is smooth and efficient.
- Input forms for registration, login, and sending emails work without noticeable delay.
- The layout is simple, making it easy for users to understand and operate.

Backend Processing:

- CRUD operations (Create, Read, Update, Delete) for users and emails are executed correctly without errors.
- Login authentication system works properly with session management.
- Data validation ensures that only valid inputs are stored in the database.
- Backend processing using Python enables smooth communication between frontend and database.

Database Operations:

- SQL queries are executed efficiently due to proper schema design.
- Data insertion, retrieval, updating, and deletion operations are consistent and reliable.
- Foreign key relationship between Users and Emails ensures proper data linkage.
- Database maintains integrity and consistency even with multiple operations.

Security Measures:

- Basic authentication restricts unauthorized access to the system.
- Input validation prevents invalid or empty data entries.
- Unique email constraint ensures no duplicate user accounts.
- Session management provides secure access during login sessions.

Scalability (Initial Observation):

- The system handled multiple users and email records efficiently during testing.
- Performance remained stable even when the number of records increased.
- The system can be further improved using indexing and query optimization for large-scale usage.

5.2 OBSERVATIONS AND INTERPRETATIONS

User Experience:

Positive Feedback:

- The system is simple and user-friendly.
- Users can easily send, receive, and delete emails.
- The interface requires minimal technical knowledge.
- Inbox display improves accessibility of messages.

Challenges Observed:

- Confirmation messages can be added for delete operations.
- Advanced search functionality can improve usability.
- User interface can be enhanced with additional features.

Admin Insights:

- The system allows easy monitoring of user and email data.
- The database structure provides organized storage of information.
- Additional tracking features can be implemented for better control.

Functional Stability:

- All modules such as registration, login, sending emails, inbox viewing, and deletion function correctly.
- No major errors or system crashes were observed during testing.
- Proper error handling ensures smooth system operation.

Visual and Layout Aspects:

- The interface is clean and well-structured.
- Streamlit provides a simple and interactive layout.
- Data presentation is clear and easy to read.
- The design supports smooth user interaction across different devices.

Additional Observations:

- The system ensures efficient data handling with minimal delay.
- Proper database design helps maintain consistency and integrity.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF OUTCOMES

The performance of the Mini Email System was evaluated based on key parameters such as system efficiency, accuracy of database operations, real-time responsiveness, and overall user experience. The system was tested under different scenarios to ensure reliability and correctness of operations.

The application successfully implements all essential email operations including user registration, login authentication, sending emails, receiving messages, and deleting emails. All CRUD operations function correctly and ensure smooth data handling without errors. The system effectively demonstrates the practical implementation of database management concepts in a real-time application.

The use of MySQL as the database provides an efficient and structured data storage solution. Proper normalization, use of primary and foreign keys, and well-defined schema design help maintain data integrity and eliminate redundancy. This ensures that the data remains consistent and reliable during multiple operations.

The Streamlit-based user interface is interactive, simple, and easy to navigate. It allows users to perform operations without requiring technical expertise. The design of the interface enhances usability and provides a smooth user experience.

Data handling within the system is efficient, with fast and accurate execution of SQL queries for storing and retrieving email information. The system performs consistently without crashes or unexpected behavior, and proper validation and error handling ensure stable operation.

Overall, the project successfully achieves its objective of developing a simple and efficient Mini Email System. It reduces manual effort and provides a basic platform for email communication using database concepts.

The system has certain limitations, such as the use of basic authentication without encryption, lack of support for attachments and real-time notifications, and suitability mainly for small-scale applications. These limitations indicate the scope for further improvements and enhancements.

6.2 FUTURE SCOPE AND ENHANCEMENTS

While the Mini Email System meets its primary objectives, there is significant scope for further improvement and expansion. Enhancing the system with additional features can make it more efficient, secure, and suitable for real-world applications.

Advanced features such as email attachments can be implemented to allow users to send files along with messages. Real-time notification mechanisms can be added to inform users when new emails are received. Security can be improved by implementing encryption techniques and secure authentication methods.

The system can be extended by introducing spam filtering mechanisms to detect and block unwanted emails. Cloud-based deployment can be adopted to enable remote access and support multiple users. Advanced search and filtering options can be included to improve email retrieval efficiency.

Further enhancements may include the development of a mobile application to increase accessibility, implementation of automated backup and recovery mechanisms to ensure data safety, and creation of an analytics dashboard to provide insights into system usage.

The modular design of the Mini Email System allows these enhancements to be integrated easily without major changes to the existing structure. This ensures scalability and adaptability of the system for future requirements and technological advancements.

APPENDICES

APPENDIX A

SOURCE CODE

This section presents key source code snippets from the Library Management System developed using Python (Streamlit) and SQLite database. The system follows a modular approach for better organization and maintainability.

```
import streamlit as st
import mysql.connector

# DB CONNECTION
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="s@kthi_0103",
    database="email_system"
)
cursor = db.cursor()

st.set_page_config(page_title="Mini Email System", layout="wide")

# ------------------- LOGIN PAGE -------------------
def login_page():
    st.title("✉ Mini Email System")

    choice = st.radio("Select", ["Login", "Signup"])

    # SIGNUP
```

```
if choice == "Signup":
    st.subheader("Create Account")

    name = st.text_input("Name")
    email = st.text_input("Email")
    password = st.text_input("Password", type="password")

    if st.button("Signup"):
        cursor.execute("SELECT * FROM Users WHERE email=%s", (email,))
        if cursor.fetchone():
            st.error("Email already exists!")
        else:
            cursor.execute(
                "INSERT INTO Users (name,email,password) VALUES (%s,%s,%s)",
                (name,email,password)
            )
            db.commit()
            st.success("Account created!")

# LOGIN
else:
    st.subheader("Login")

    email = st.text_input("Email")
    password = st.text_input("Password", type="password")

    if st.button("Login"):
        cursor.execute(
            "SELECT user_id,name,email FROM Users WHERE email=%s AND
password=%s",
            (email,password)
```

```
        )
        user = cursor.fetchone()

        if user:
            st.session_state.user_id = user[0]
            st.session_state.name = user[1]
            st.session_state.email = user[2]
            st.success("Login successful!")
            st.rerun()
        else:
            st.error("Invalid login")

# ------------------ HOME PAGE ------------------
def home_page():

    # Sidebar Profile
    st.sidebar.title("👤 Profile")
    st.sidebar.write("Name: " + st.session_state.name)
    st.sidebar.write("Email: " + st.session_state.email)

    st.sidebar.markdown("---")

    option = st.sidebar.radio("Menu", [
        "📥 Inbox",
        "✉️ Send Email",
        "🗑️ Delete Email",
        "🚪 Logout"
    ])

    st.title("Welcome " + st.session_state.name)
```

```
# ------------------ INBOX ------------------
if option == "📥 Inbox":
    st.subheader("Inbox")

    cursor.execute(
        "SELECT email_id,subject,message FROM Emails WHERE receiver_email=%s AND
is_deleted=FALSE",
        (st.session_state.email,)
    )

    emails = cursor.fetchall()

    if emails:
        for row in emails:
            st.write(f"📥 ID: {row[0]}")
            st.write(f"Subject: {row[1]}")
            st.write(f"Message: {row[2]}")
            st.write("---")
    else:
        st.info("No emails")

# ------------------ SEND EMAIL ------------------
elif option == "✉ Send Email":
    st.subheader("Send Email")

    receiver = st.text_input("Receiver Email").strip()
    subject = st.text_input("Subject")
    message = st.text_area("Message")

    if st.button("Send"):
```

```
        if receiver == "":
            st.error("Enter receiver email")
        else:
            cursor.execute(
                "INSERT INTO Emails (sender_id, receiver_email, subject, message) VALUES
(%s,%s,%s,%s)",
                (st.session_state.user_id, receiver, subject, message)
            )
            db.commit()
            st.success("Email sent!")

    # ------------------ DELETE EMAIL ------------------
    elif option == "🗑 Delete Email":
        st.subheader("Delete Email")

        eid = st.text_input("Enter Email ID")

        if st.button("Delete"):
            cursor.execute(
                "UPDATE Emails SET is_deleted=TRUE WHERE email_id=%s",
                (eid,)
            )
            db.commit()
            st.success("Email deleted")

    # ------------------ LOGOUT ------------------
    elif option == "🚪 Logout":
        del st.session_state.user_id
        st.success("Logged out")
        st.rerun()
```

```
# ------------------- MAIN -------------------
if "user_id" not in st.session_state:
    login_page()
else:
    home_page()
```

APPENDIX B – Screenshots

Figure B.1 User Registration Page

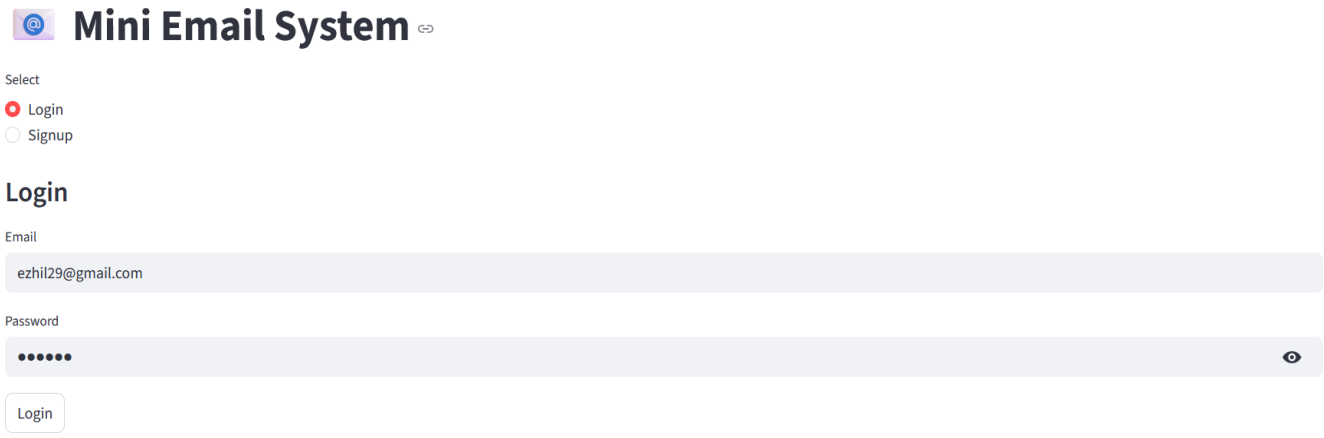

Figure B.2 LOGIN PAGE

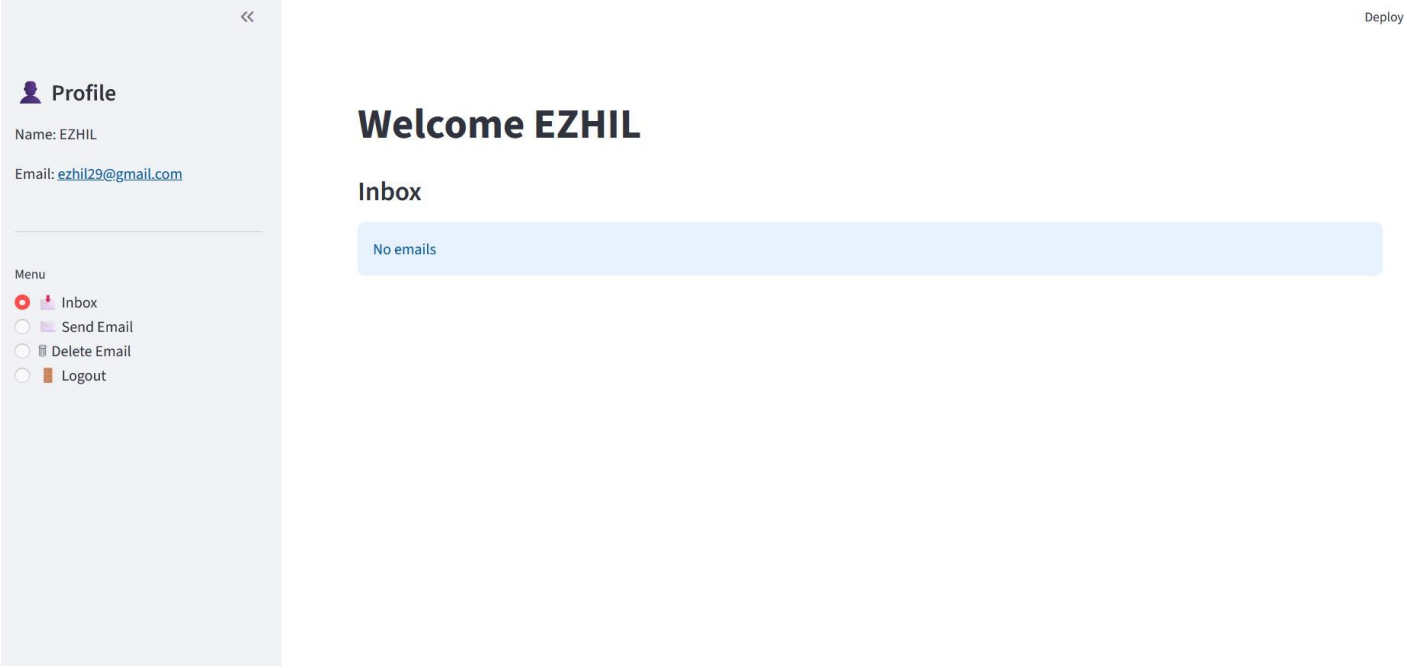

Figure B.3 Inbox Display

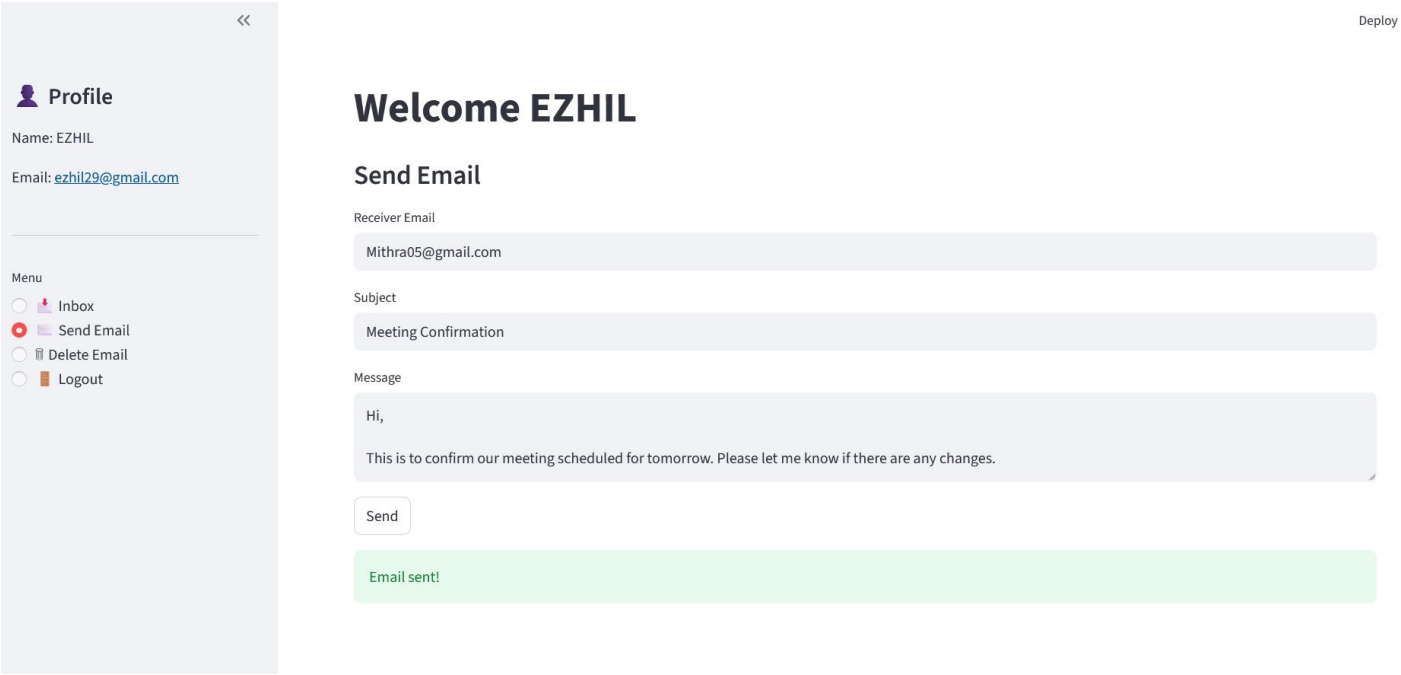

Figure B.4 Send Email Interface

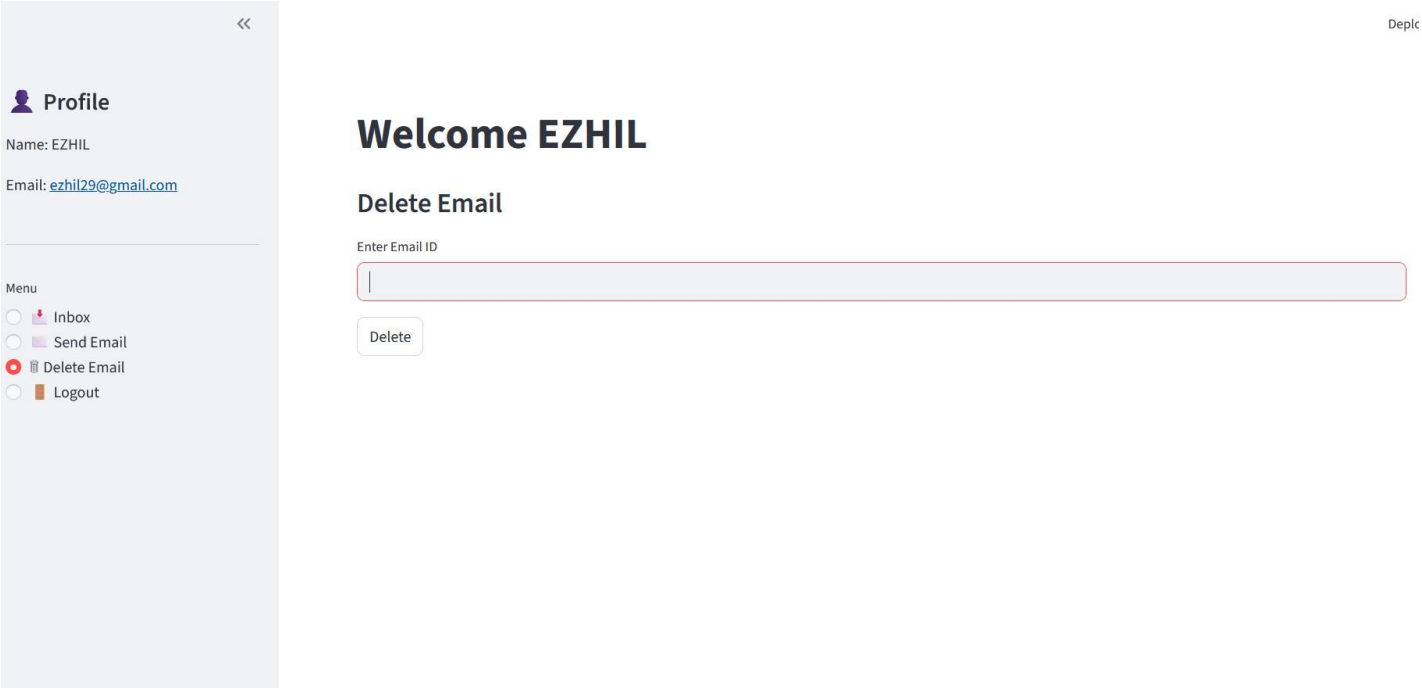

Figure B.5 Delete Email Module

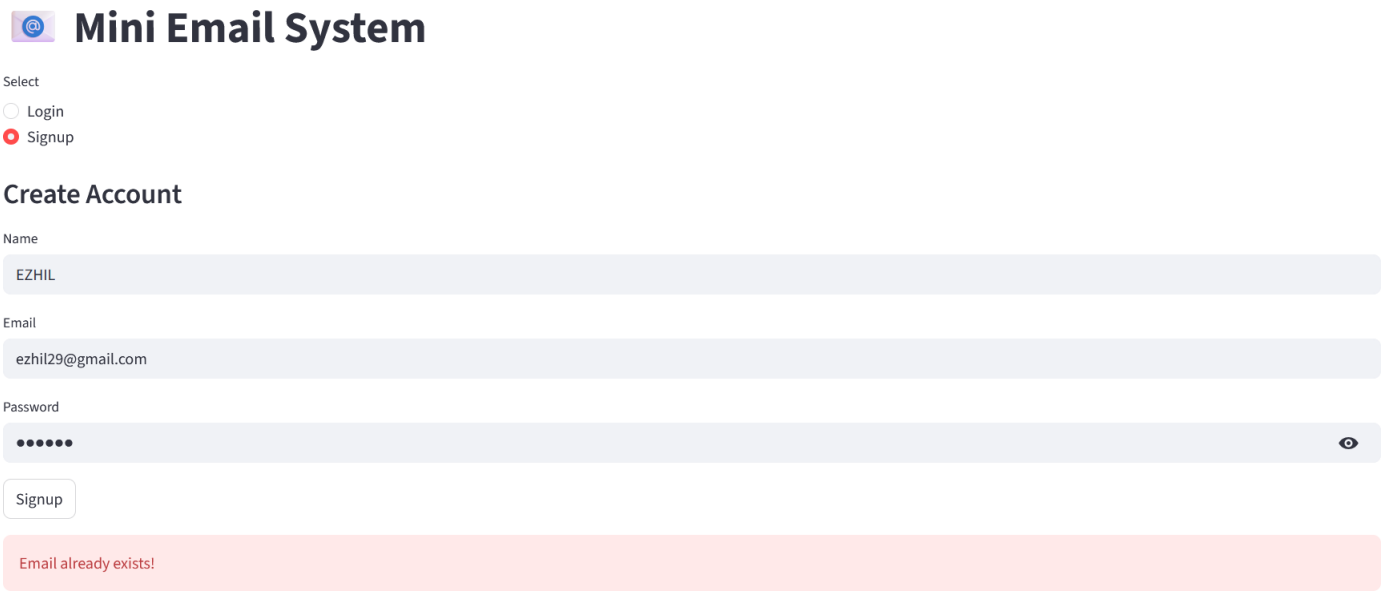

Figure B.6 Duplicate Email Validation during User Registration

REFERENCES

[1] Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, Pearson Education, 7th Edition.

[2] GeeksforGeeks, Database Management System Tutorials and Concepts for Beginners and Advanced Learners.

[3] Korth, H. F., Silberschatz, A., & Sudarshan, S., *Database System Concepts*, McGraw-Hill, 6th Edition.

[4] MySQL Documentation, MySQL Reference Manual for Database Design, Queries, and Administration.

[5] Oracle Corporation, Database Design Concepts and Relational Database Architecture Guide.

[6] Python Software Foundation, Python Programming Language Documentation and Developer Guide.

[7] Ramakrishnan, R., & Gehrke, J., *Database Management Systems*, McGraw-Hill, 3rd Edition.

[8] Streamlit Documentation, Streamlit User Guide for Building Interactive Web Applications using Python.

[9] TutorialsPoint, Comprehensive SQL Tutorial Covering Database Queries, Joins, and Data Manipulation Techniques.

[10] W3Schools, SQL and Database Tutorials Including Practical Examples for Beginners and Advanced Users.